Manage Machine Learning Experiments with Amazon SageMaker Model Tracking Capability

To organize, find, and evaluate machine leaning model experiments, use Amazon SageMaker model tracking capabilities. Developing models typically requires extensive experimenting with different datasets, algorithms, and parameter values. Using the model tracking capability, you can search, filter and sort through hundreds and possibly thousands of experiments using model attributes such as parameters, metrics and tags. This helps you find the best model for your use case quickly.

Amazon SageMaker model tracking capability can be used to:

- Find, organize, or evaluate training jobs using properties, hyperparameters, performance metrics, or any other metadata.
- Find the best performing model by ranking the results of training jobs and models based on metrics, such as training loss or validation accuracy.
- Trace the lineage of a model back to the training job and its related resources, such as the training datasets.

Sample Notebooks that Manage ML Experiments with Amazon SageMaker Model Tracking Capability

For a sample notebook that uses Amazon SageMaker model tracking capability to manage ML experiments, see Managing ML Experimentation using Amazon SageMaker Model Tracking Capability. For instructions on how to create and access Jupyter notebook instances that you can use to run the example in Amazon SageMaker, see Use Notebook Instances (p. 36). After you have created a notebook instance and opened it, choose the **SageMaker Examples** tab to see a list of all of the Amazon SageMaker samples. The notebook managing ML experiments is located in the **Advanced Functionality** section. To open a notebook, choose its **Use** tab and choose **Create copy**. If you have questions, post them on our developer forum.

Topics

- Use Model Tracking to Find, Organize, and Evaluate Training Jobs (Console) (p. 434)
- Use Model Tracking to Find and Evaluate Training Jobs (API) (p. 436)
- Verify the Contents of Your Training Jobs (p. 438)
- Trace the Lineage of your Models (p. 438)

Use Model Tracking to Find, Organize, and Evaluate Training Jobs (Console)

To create and test a model, you need to conduct experiments. You can perform an experiment to test different algorithms, tune hyperparameters, or use different datasets. Typically, you study the effect of the changes made in these experiments on the performance of a model. You can organize these experiments by tagging them with key/ value pairs that are seaerchable.

To find a specific training job, model, or resource, use Amazon SageMaker model tracking to search on keywords in any items that are searchable. Searchable items include training jobs, models, hyperparameters, metadata, tags, and URLs. You can use model tracking with tags to help organize your training jobs. To refine your tracking results, you can search using multiple criteria.

To choose the optimal model for deployment, you need to evaluate how they performed against one or more metrics. You can use model tracking results to list, sort, and evaluate the performance of the models in your experiments.

Topics

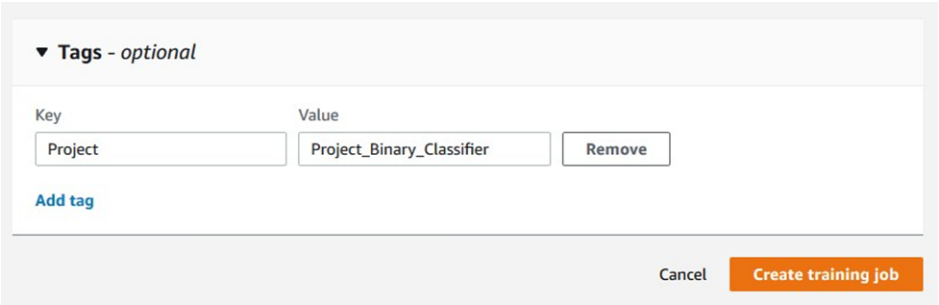
- Use Tags to Track Training Jobs (Console) (p. 434)
- Find Training Jobs (Console) (p. 435)
- Evaluate Models Returned by a Search (Console) (p. 435)

Use Tags to Track Training Jobs (Console)

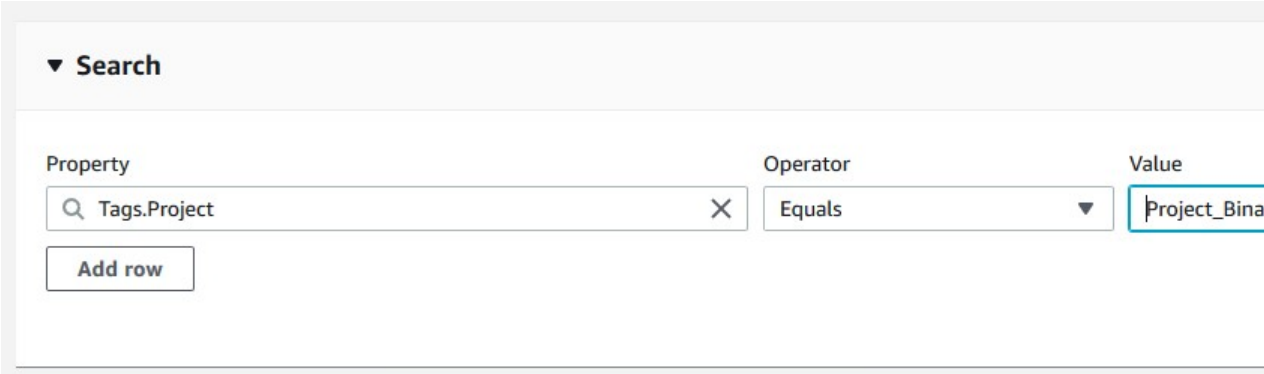
You can use tags as search criteria. To group training jobs, create tags with descriptive keys and value. For example, create tag keys for: project, owner, customer, and industry.

Add tags to training job and search for taged jobs (console)

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker.
2. In the left navigation pane, choose **Training jobs** and select **Create training job**.
3. Scroll down to the bottom of the page to enter the Key and Value for the tag.



4. To add more tags to the search, choose **Add tag** and add a another tag key-value pair for each new tag that you want to add.
5. After you have trained models that have been tagged, you can search for the models that had them added. In the left navigation pane, choose **Search**.
6. For **Property**, enter a tag key and a tag value.



When you use tags in a search, in the results, the key is a column name and the values are entries in rows.

Find Training Jobs (Console)

To use Amazon SageMaker model tracking capability (console)

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker.
2. In the left navigation pane, choose **Search**.
3. For **Resource type**, choose **Training jobs**.
4. To add a parameter, for **Parameters**, provide the following:
   a. Enter a parameter in the search box and choose a parameter type, for example **TrainingJobName**.
   b. Choose the conditional operation to use. This sets the condition that values must satisfy to be included in a search result. For numeral values, use operators such as **is equals to**, **lesser than**, or **or greater than**. For text-based values use operators, such as **equals to** or **contains**.
   c. Enter the value for the parameter.
5. (optional) To refine your search, add additional search criteria, choose **Add parameter** and enter the parameter values. When you provide multiple parameters, Amazon SageMaker includes all parameters in the search.
6. Choose **Search**.

Evaluate Models Returned by a Search (Console)

To evaluate different models, find their metrics with a search. To highlight metrics, adjust the view to show only metrics and important hyperparameters.

To change the viewable metadata, hyperparameters, and metrics:

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker.
2. In the left navigation pane, choose **Search** and run a search on training jobs for relevant parameters. The results are displayed in a table.

Results: Training jobs

HyperParameter mini_batch_size	HyperParameter predictor_type	Metric train:binary_f_beta	Metric train:progress	Metric train:objective_los
300	binary_classifier	0.966639518737793	100	0.02381423674523
100	binary_classifier	0.9652714133262634	100	0.0235049128532
200	binary_classifier	0.9647442698478699	100	0.0232598073780

3. After performing a search, choose the cog icon at the search results table to show the preferences window.
4. To show or hide a **Hyperparameter** or **Metric**, use its toggle switch.
5. To update the view after making changes, choose **Update view**.

After viewing metrics and important hyperparameters, you can compare and contrast the result. From there, you can choose the best model to host or investigate the models that are performing poorly.

Use Model Tracking to Find and Evaluate Training Jobs (API)

You can also use Search (p. 819) to the find and evaluate training jobs or to get suggestions for items used in experiments that are searchable.

Topics

- Use Search to Find Training Jobs Tagged with Specific Values (API) (p. 436)
- Evaluate Models (API) (p. 436)
- Get Suggestions for a Search (API) (p. 437)

Use Search to Find Training Jobs Tagged with Specific Values (API)

To use Amazon SageMaker model tracking capability, create a search parameter, `search_params`, then use the search function found in the `smclient` of the AWS SDK for Python (Boto 3).

The following example shows how to use search using the API:

```
import boto3

search_params={
   "MaxResults": 10,
   "Resource": "TrainingJob",
   "SearchExpression": {
      "Filters": [{
            "Name": "Tags.Project",
            "Operator": "Equals",
            "Value": "Project_Binary_Classifier"
         }]},
  "SortBy": "Metrics.train:binary_classification_accuracy",
  "SortOrder": "Descending"
}

smclient = boto3.client(service_name='sagemaker')
results = smclient.search(**search_params)
```

Evaluate Models (API)

To evaluate different models, see the metrics in a search result. To evaluate models using the AWS SDK for Python (Boto 3), create a table and plot it.

The following example shows how to use model tracking capability to evaluate models and to display the results in a table:

```
import pandas

headers=["Training Job Name", "Training Job Status", "Batch Size", "Binary Classification
 Accuracy"]
rows=[]
for result in results['Results']:
    trainingJob = result['TrainingJob']
    metrics = trainingJob['FinalMetricDataList']
    rows.append([trainingJob['TrainingJobName'],
     trainingJob['TrainingJobStatus'],
     trainingJob['HyperParameters']['mini_batch_size'],
     metrics[[x['MetricName'] for x in
     metrics].index('train:binary_classification_accuracy')]['Value']
    ])

df = pandas.DataFrame(data=rows,columns=headers)

from IPython.display import display, HTMLdisplay(HTML(df.to_html()))
```

Get Suggestions for a Search (API)

To get suggestions for a search, use GetSearchSuggestions (p. 759) in the API.

The following code example for AWS SDK for Python (Boto 3) shows a `get_search_suggestions` request for items containing "linear":

```
search_suggestion_params={
   "Resource": "TrainingJob",
   "SuggestionQuery": {
      "PropertyNameQuery": {
         "PropertyNameHint": "linear"
      }
   }
```

```
}
```

The following code shows an example of an expected response for `get_search_suggestions`:

```
{
        'PropertyNameSuggestions': [{'PropertyName': 'hyperparameters.linear_init_method'},
                    {'PropertyName': 'hyperparameters.linear_init_value'},
                    {'PropertyName': 'hyperparameters.linear_init_sigma'},
                    {'PropertyName': 'hyperparameters.linear_lr'},
                    {'PropertyName': 'hyperparameters.linear_wd'}]
}
```

After you get the search suggestions, you can use one of the property names in a search.

Verify the Contents of Your Training Jobs

You can use Amazon SageMaker model tracking capability to verify which datasets were used in training, where the holdout datasets were used, and other details about training jobs. Use it, for example, if you need to verify that a dataset was used in a training job for an audit or to verify compliance.

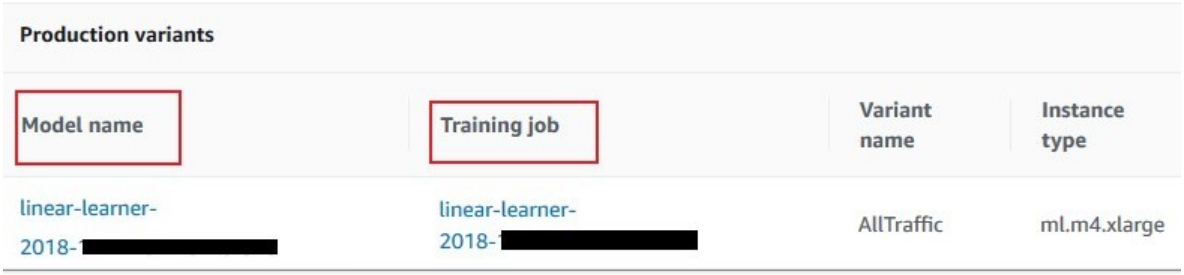
To check if a holdout dataset or any other dataset was used in a training job, search for its Amazon S3 URL. Amazon SageMaker model tracking uses the dataset as a search parameter and lists the training jobs that used the dataset. If your search result is empty, it means that the dataset was not used in a training job.

Trace the Lineage of your Models

You can use Amazon SageMaker model tracking to trace information about the lineage of training jobs and model resources related to it, including the dataset, algorithm, hyperparameters, and metrics used. For example, if you find that the performance of a hosted model has declined, you can review its training job and the resources it used to determine what is causing the problem. This investigation can be done from the console or by using the API.

Use Single-click on the Amazon SageMaker Console to Trace the Lineage of Your Models (Console)

In the left navigation pane of the Amazon SageMaker, choose **Endpoints**, and select the relevant endpoint from the list of your deployed endpoints. Scroll to **Endpoint Configuration Settings**, which lists all the model versions deployed at the endpoint. Here you have access to a hyperlink to the Model Training Job that created that model in the first place. For an example that used a linear-learner model, you would see:



Use Code to Trace the Lineage of Your Models (API)

To trace a model's lineage, you need to obtain the model's name, then use it to search for training jobs.

The following example shows how to use model tracking capability to trace a model's lineage using the API:

```
# Get the name of model deployed at endpoint
endpoint_config = smclient.describe_endpoint_config(EndpointConfigName=endpointName)
model_name = endpoint_config['ProductionVariants'][0]['ModelName']

# Get the model's name
model = smclient.describe_model(ModelName=model_name)

# Search the training job by Amazon S3 location of model artifacts
search_params={
   "MaxResults": 1,
   "Resource": "TrainingJob",
   "SearchExpression": {
      "Filters": [
         {
            "Name": "ModelArtifacts.S3ModelArtifacts",
            "Operator": "Equals",
            "Value": model['PrimaryContainer']['ModelDataUrl']
         }]},
}
results = smclient.search(**search_params)
```

After you find your training job, you can investigate the resources used to train the model.